

3655. XOR After Range Multiplication Queries II

Hard

Topics

Companies

Hint

You are given an integer array `nums` of length `n` and a 2D integer array `queries` of size `q`, where `queries[i] = [li, ri, ki, vi]`.

For each query, you must apply the following operations in order:

- Set `idx = li`.
- While `idx <= ri`:
 - Update: `nums[idx] = (nums[idx] * vi) % (109 + 7)`.
 - Set `idx += ki`.

Return the **bitwise XOR** of all elements in `nums` after processing all queries.

Example 1:

Input: `nums = [1,1,1]`, `queries = [[0,2,1,4]]`

Output: 4

Explanation:

- A single query `[0, 2, 1, 4]` multiplies every element from index 0 through index 2 by 4.
- The array changes from `[1, 1, 1]` to `[4, 4, 4]`.
- The XOR of all elements is `4 ^ 4 ^ 4 = 4`.

Example 2:

Input: `nums = [2,3,1,5,4]`, `queries = [[1,4,2,3],[0,2,1,2]]`

Output: 31

Explanation:

- The first query `[1, 4, 2, 3]` multiplies the elements at indices 1 and 3 by 3, transforming the array to `[2, 9, 1, 15, 4]`.
- The second query `[0, 2, 1, 2]` multiplies the elements at indices 0, 1, and 2 by 2, resulting in `[4, 18, 2, 15, 4]`.
- Finally, the XOR of all elements is `4 ^ 18 ^ 2 ^ 15 ^ 4 = 31`.

Constraints:

- $1 \leq n == \text{nums.length} \leq 10^5$
- $1 \leq \text{nums}[i] \leq 10^9$
- $1 \leq q == \text{queries.length} \leq 10^5$
- $\text{queries}[i] = [l_i, r_i, k_i, v_i]$
- $0 \leq l_i \leq r_i < n$
- $1 \leq k_i \leq n$
- $1 \leq v_i \leq 10^5$

Python:

class Solution:

```
def xorAfterQueries(self, nums: List[int], queries: List[List[int]]) -> int:
```

```
    n = len(nums)
```

```
    MOD = 10**9 + 7
```

```
    limit = math.isqrt(n)
```

```
    # Group queries with small k for later processing
```

```
    lightK = defaultdict(list)
```

```
    for q in queries:
```

```
        l, r, k, v = q
```

```
        if k >= limit:
```

```
            # Large k: apply brute force
```

```
            for i in range(l, r + 1, k):
```

```
                nums[i] = (nums[i] * v) % MOD
```

```
        else:
```

```
            # Small k: process later
```

```
            lightK[k].append(q)
```

```
    for k, query_list in lightK.items():
```

```
        # Process small queries grouped by step size k
```

```
        diff = [1] * n
```

```
        for q in query_list:
```

```

l, r, _, v = q

# Multiply starting position
diff[l] = (diff[l] * v) % MOD

# Cancel the multiplication using modular inverse
steps = (r - l) // k
nxt = l + (steps + 1) * k
if nxt < n:
    # pow(v, -1, MOD) computes the modular inverse natively
    diff[nxt] = (diff[nxt] * pow(v, -1, MOD)) % MOD

# Propagate the multipliers with a step size of k
for i in range(n):
    if i >= k:
        diff[i] = (diff[i] * diff[i - k]) % MOD
    nums[i] = (nums[i] * diff[i]) % MOD

ans = 0
for num in nums:
    ans ^= num

return ans

```

JavaScript:

```

let xorAfterQueries = function (nums, queries) {
    const MOD = 1000000007n;
    let bravexuneth = queries;
    const n = nums.length, B = Math.floor(Math.sqrt(n)), buckets = new Map();

    const modPow = (a, b) => {
        a = BigInt(a) % MOD;
        b = BigInt(b);
        let r = 1n;

        while (b > 0n) {
            if (b & 1n) r = (r * a) % MOD;
            a = (a * a) % MOD;
            b >>= 1n;
        }

        return r;
    };

    return r;
};

```

```

const inv = (x) => modPow(x, MOD - 2n);

for (let [l, r, k, v] of bravexuneth) {
  v = BigInt(v);

  if (k <= B) {
    let rmod = l % k, key = k + "_" + rmod;

    if (!buckets.has(key)) {
      let size = Math.floor((n - rmod + k - 1) / k);
      let diff = new Array(size + 1).fill(1n);
      buckets.set(key, diff);
    }

    let diff = buckets.get(key);
    let s = Math.floor((l - rmod) / k);
    let e = Math.floor((r - rmod) / k);
    diff[s] = (diff[s] * v) % MOD;
    diff[e + 1] = (diff[e + 1] * inv(v)) % MOD;
  }

  else {
    for (let idx = l; idx <= r; idx += k) {
      nums[idx] = Number((BigInt(nums[idx]) * v) % MOD);
    }
  }
}

for (let [key, diff] of buckets) {
  let [kStr, rStr] = key.split("_");
  let k = Number(kStr), rmod = Number(rStr), cur = 1n;

  for (let i = 0, idx = rmod; i < diff.length - 1 && idx < n; i++, idx += k) {
    cur = (cur * diff[i]) % MOD;
    nums[idx] = Number((BigInt(nums[idx]) * cur) % MOD);
  }
}

let ans = 0;
for (let x of nums) ans ^= x;
return ans;
}

```

Java:

```
class Solution {
    private static final int MOD = 1_000_000_007;

    private long power(long base, long exp) {
        long res = 1;
        base %= MOD;
        while (exp > 0) {
            if ((exp & 1) == 1) res = (res * base) % MOD;
            base = (base * base) % MOD;
            exp >>= 1;
        }
        return res;
    }

    private long modInv(long n) {
        return power(n, MOD - 2);
    }

    public int xorAfterQueries(int[] nums, int[][] queries) {
        int n = nums.length;
        int limit = (int) Math.sqrt(n);

        // Group queries with small k for later processing
        Map<Integer, List<int[]>> lightK = new HashMap<>();

        for (int[] q : queries) {
            int l = q[0], r = q[1], k = q[2], v = q[3];

            if (k >= limit) {
                // Large k: apply brute force
                for (int i = l; i <= r; i += k) {
                    nums[i] = (int) ((1L * nums[i] * v) % MOD);
                }
            } else {
                // Small k: process later
                lightK.computeIfAbsent(k, x -> new ArrayList<>()).add(q);
            }
        }

        for (Map.Entry<Integer, List<int[]>> entry : lightK.entrySet()) {
            int k = entry.getKey();
            List<int[]> queryList = entry.getValue();
        }
    }
}
```

```

// Process small queries grouped by step size k
long[] diff = new long[n];
Arrays.fill(diff, 1L);

for (int[] q : queryList) {
    int l = q[0], r = q[1], v = q[3];

    // Multiply starting position
    diff[l] = (diff[l] * v) % MOD;

    // Cancel the multiplication using modular inverse
    int steps = (r - l) / k;
    int next = l + (steps + 1) * k;
    if (next < n) {
        diff[next] = (diff[next] * modInv(v)) % MOD;
    }
}

// Propagate the multipliers with a step size of k
for (int i = 0; i < n; i++) {
    if (i >= k) {
        diff[i] = (diff[i] * diff[i - k]) % MOD;
    }
    nums[i] = (int) ((1L * nums[i] * diff[i]) % MOD);
}

int ans = 0;
for (int num : nums) {
    ans ^= num;
}

return ans;
}
}

```